

Graph Traversal

□ Graph Traversal Algorithms

Graph Traversal is the systematic process of visiting every vertex in a graph.

Algorithm Method		Data Structure	Time Complexity	Primary Use Case
BFS	Breadth-First Search (Level by Level)	Queue (FIFO)	$O(V + E)$	Shortest Path in unweighted graphs.
DFS	Depth-First Search (Goes as deep as possible)	Stack (LIFO/ Recursion)	$O(V + E)$	Cycle Detection, finding connected components.
Dijkstra's	Processes nodes in order of increasing path cost .	Priority Queue	$O(E + V \log V)$	Shortest Path in weighted graphs (non-negative weights).

🔍 BFS (Breadth-First Search)

- **Principle:** Explore all immediate neighbors before moving to the next level of nodes.
- **Equivalence:** Equivalent to **Dijkstra's Algorithm** when all edge weights are 1.
- **Process Core:** Use a queue. Dequeue node \$u\$, then enqueue all its **unvisited** neighbors \$v\$.

C++ Code

```
#include <iostream>
#include <vector>
#include <queue>
#include <list>

void bfs_traversal(int V, const std::list<int> adj[], int start) {
    std::vector<bool> visited(V, false);
    std::queue<int> q;

    visited[start] = true;
    q.push(start);
    std::cout << "BFS Traversal: ";

    while (!q.empty()) {
        int u = q.front(); q.pop();
        std::cout << u << " "; // Process node u

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

```

        q.push(v);
    }
}

std::cout << std::endl;
}

// Example usage:
// int V = 4;
// std::list<int> adj[V];
// adj[0].push_back(1); adj[0].push_back(2);
// adj[1].push_back(2);
// adj[2].push_back(0); adj[2].push_back(3);
// adj[3].push_back(3);
// bfs_traversal(V, adj, 2); // Output: 2 0 3 1

```

🌲 DFS (Depth-First Search)

- **Principle:** Recursively explore one branch as far as possible, then backtrack (return) to try the next branch.
- **Process Core:** Use recursion. Mark node u as visited, then recursively call DFS on all **unvisited** neighbors v .

C++ Code

```

#include <iostream>
#include <vector>
#include <list>

// The recursive utility function (The Core)
void dfsUtil(int u, const std::list<int> adj[], std::vector<bool>& visited) {
    visited[u] = true;
    std::cout << u << " "; // Process node u

    // The key recursive call: Explore unvisited neighbors
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfsUtil(v, adj, visited);
        }
    }
}

// The wrapper function to start the traversal
void dfs_traversal(int V, const std::list<int> adj[], int start) {
    std::vector<bool> visited(V, false);
    std::cout << "DFS Traversal: ";
    dfsUtil(start, adj, visited);
    std::cout << std::endl;
}

```

```
// Example usage:
// int V = 4;
// std::list<int> adj[V];
// adj[0].push_back(1); adj[0].push_back(2);
// adj[1].push_back(3);
// adj[2].push_back(3);
// dfs_traversal(V, adj, 0); // Output: 0 1 3 2 (Order depends on list
```

Topological Sort (Kahn's Algorithm)

- **Goal:** Create a linear ordering of vertices in a **Directed Acyclic Graph (DAG)** where u comes before v if there is an edge $u \rightarrow v$.
- **Kahn's Algorithm:** The **BFS-based** approach that relies on **in-degrees**.
- **Process:**
 1. Calculate **in-degree** for all nodes.
 2. Enqueue all nodes with **in-degree = 0**.
 3. Dequeue a node u , add to result, and decrement the in-degree of its neighbors v .
 4. If v 's in-degree drops to 0, enqueue v .

C++ Code (Kahn's Algorithm Core)

```
#include <iostream>
#include <vector>
#include <queue>
#include <list>
#include <algorithm> // For std::find, though not strictly needed here

std::vector<int> topologicalSortKahn(int V, const std::list<int> adj[])
{
    std::vector<int> in_degree(V, 0);
    std::queue<int> q;
    std::vector<int> result;

    // 1. Compute in-degrees (O(V+E))
    for (int u = 0; u < V; ++u)
        for (int v : adj[u]) in_degree[v]++;

    // 2. Initialize queue with nodes having 0 in-degree
    for (int i = 0; i < V; ++i)
        if (in_degree[i] == 0) q.push(i);

    // 3. Process nodes
    while (!q.empty()) {
        int u = q.front(); q.pop();
        result.push_back(u);

        // 4. Reduce in-degree of neighbors
        for (int v : adj[u]) {
            in_degree[v]--;
            if (in_degree[v] == 0) q.push(v);
        }
    }

    return result;
}
```

```
        }
    }

    // Check for a cycle
    if (result.size() != V) {
        std::cerr << "Error: Graph has a cycle, topological sort is not\n";
        return {};
    }

    return result;
}
```